

DevPulse

PRD v3.0 — Full Consolidated Master PRD

Agentic Developer Change Intelligence & Migration Decision System

DOCUMENT TYPE · Product Requirements Document

VERSION · v3.0 — Full Consolidated Master PRD

STATUS · Approved PRD Candidate

DATE · April 2026

SCOPE · Solo builder · Senior/Staff-leaning DS/AI systems · Agentic AI readiness

POSITIONING · Production-simulated · Non-production · Evidence-backed · LLM-last · Agentic orchestration

DevPulse v3.0 is a production-simulated developer change-intelligence system that combines version-aware hybrid retrieval, deterministic conflict detection, grounded LLM-last synthesis, SAFE/RISKY/BLOCKED migration decision reports, and bounded agentic orchestration over dependency migration goals. The system produces a 37-day simulated operating lifecycle, 32 evidence artifacts, 19 failure/recovery scenarios, and auditable traces across both query-level and goal-level workflows.

Sidharth Kriplani

Solo-built · Non-production · Production-simulated · April 2026

Table of Contents

1. Executive Summary
2. Why DevPulse v3.0 Exists
3. Product Positioning
4. Production-Simulated System Definition
5. System Architecture Overview
6. Query Mode — Version-Aware Migration Decision Core
7. Goal Mode — Agentic Orchestration Layer
8. Data Model and Schemas
9. Retrieval, Version Correctness, and Conflict Detection
10. Migration Decision Report Design
11. Observability and Audit Design
12. LLM-Last Orchestration Rules
13. 37-Day Simulated Operating Lifecycle
14. Evidence Artifacts — EA-01 to EA-32
15. Failure and Recovery Scenarios — F-01 to F-19
16. Evaluation Design and Acceptance Gates
17. Demo Narrative
18. Must-Have / Should-Have / Optional / Never-Cut
19. Final Deliverable Checklist
20. Resume and Interview Positioning
21. Red-Team Review
22. Recommendation Before Execution Planning
23. Honest Boundary Statement
24. Revision History

1. Executive Summary

DevPulse v3.0 is a production-simulated agentic developer change-intelligence and migration decision system. It helps engineering teams reason safely about external dependency changes by combining version-aware retrieval, deterministic conflict detection, LLM-last grounded synthesis, SAFE/RISKY/BLOCKED migration decision reports, and bounded goal-level agentic orchestration.

The Problem

Engineering teams working with evolving external dependencies face a compounding risk: the guidance they trust may be version-wrong. A migration guide for v2.x describes an API that was removed in v3.x. A reference doc shows a method signature that changed three minor versions ago. A changelog says deprecated; the reference doc still shows it as current. Generic documentation chatbots retrieve the most semantically similar answer regardless of version — which means they confidently return wrong-version guidance. That is worse than no answer.

DevPulse is built around one non-negotiable correctness property: a system that retrieves the right answer for the wrong version must be treated as a failure, not a partial success. Every architectural decision flows from this principle.

Two Operating Modes

Mode	Entry	Output
Query Mode	Single migration or developer query	Version-aware retrieval → conflict detection → SAFE/RISKY/BLOCKED migration decision report
Goal Mode	High-level migration goal + dependency manifest	Decomposed task plan → per-task Query Mode execution → bounded recovery → aggregate migration plan

Query Mode Flow

Single query → QueryParser → VersionExtractor → ComplexityRouter → HybridRetrieval (BM25 + pgvector + RRF) → ConflictDetector (9 types) → MigrationReporter (SAFE/RISKY/BLOCKED) → optional LLMSynthesizer (conditional, LLM-last) → PostGenValidation → CitationAssembler → response.

Goal Mode Flow

Goal text + dependency manifest → GoalParser → DependencyDeltaDetector → TaskPlanner/TaskQueue → TaskExecutor loop (calls Query Mode pipeline per task) → RecoveryDecider (rule-based, retry cap = 2) → PlanSummaryReporter → aggregate SAFE/RISKY/BLOCKED plan with staged migration recommendation.

The Goal Mode agentic layer does not replace Query Mode. TaskExecutor calls the full Query Mode pipeline for every dependency migration task. Every version pre-filter, conflict check, citation, and audit log from Query Mode applies at the task level. The LLM never plans, routes, detects conflicts, assembles citations, or decides recovery. All planning, routing, and recovery are deterministic.

Why This Is Portfolio-Worthy

- **Version correctness as a hard constraint** — not a soft preference. wrong_version_answer_rate target = 0.0.
- **LLM-last design** — the LLM synthesises only already-retrieved, already-validated evidence. It never plans or decides.
- **Deterministic conflict detection** — 9 conflict types checked via metadata, not model reasoning.
- **Bounded agentic orchestration** — goal decomposition, multi-task execution, retry-capped recovery, aggregate verdicts.
- **37-day simulated lifecycle** — 32 evidence artifacts, 19 failure/recovery scenarios, full eval harness.
- **Honest production discipline** — every claimed behavior maps to a verifiable artifact. Nothing is asserted without evidence.

2. Why DevPulse v3.0 Exists

The Hiring Signal Problem

A strong portfolio project must signal not just that the builder can implement AI systems, but that they understand the failure modes that matter in production. Two hiring audiences now matter: traditional Senior DS/AI and ML engineering panels, which reward retrieval correctness, evaluation rigor, LLM bounding, and production observability — and AI-forward companies, pre-IPO teams, and applied AI builders, who additionally want evidence that the builder can construct systems that pursue goals through visible plan-act-verify-recover loops.

DevPulse v3.0 is designed to satisfy both audiences simultaneously. The Query Mode core demonstrates retrieval engineering, deterministic conflict reasoning, and evaluation discipline. The Goal Mode agentic layer demonstrates goal decomposition, bounded orchestration, and deterministic recovery — without sacrificing the correctness philosophy of the core system.

Hiring Audience	Signal DevPulse Provides
Applied ML / DS	Retrieval correctness, evaluation rigor, LLM bounding, version-safety
Platform / Infra engineering	Operationally trustworthy, version-safe, fully auditable system
Product DS	Real pain scoped, success metrics defined, scope killed ruthlessly
Senior IC	End-to-end: design, build, measure, operate, explain tradeoffs
AI-forward CEO / CTO ★	Can build systems that pursue goals, not just answer queries
Agentic AI / AI platform ★	Goal decomposition, orchestration, recovery, trace visibility
Applied AI / pre-IPO ★	AI system ownership — not AI usage

Why Not a New Project?

The agentic layer extends — not replaces — the existing correctness infrastructure. TaskExecutor strictly calls the Query Mode pipeline per task. No new retrieval system. No new conflict detection. No new synthesis rules. Three to four new orchestration components on top of eleven verified pipeline layers. The portfolio remains four pillars; DevPulse becomes one project with two modes and two hiring signals.

3. Product Positioning

DevPulse v3.0 is a production-simulated agentic developer change-intelligence and migration decision system that helps engineering teams reason safely about external dependency changes by combining version-aware retrieval, deterministic conflict detection, LLM-last grounded synthesis, SAFE/RISKY/BLOCKED migration reports, and bounded goal-level agentic orchestration over dependency migration tasks.

What DevPulse IS

- Infrastructure for safe external dependency change adoption
- A version-aware change-intelligence system over API docs, changelogs, and migration guides
- A migration decision-support tool that produces SAFE / RISKY / BLOCKED verdicts with evidence
- A bounded agentic orchestration system that decomposes migration goals into executable tasks
- A production-simulated evidence portfolio demonstrating end-to-end AI system design and discipline
- A system that prevents wrong-version guidance from being trusted

What DevPulse IS NOT

- A generic documentation chatbot
- A free-form agent where the LLM plans, routes, or makes recovery decisions
- A prototype that only works on happy-path inputs
- A system where version correctness is assumed rather than enforced
- A system claiming real production traffic, real users, or real deployment
- A live package registry integration

4. Production-Simulated System Definition

A production-simulated system is one not deployed to real users but built, operated, and evaluated as if it were. Every production risk is scripted and handled. Every evidence artifact is real output from the running system. Nothing is claimed without a verifiable artifact.

Category	Definition in DevPulse
REAL	Ingestion pipeline · metadata extraction · structural chunking · BM25 + pgvector index build · hybrid RRF retrieval · version pre-filtering · conflict detection · citation assembly · audit logging · eval harness execution · migration report generation · GoalParser · DependencyDeltaDetector · TaskPlanner · TaskExecutor · RecoveryDecider · PlanSummaryReporter

Category	Definition in DevPulse
SIMULATED / CONTROLLED	Documentation corpus (public API docs or controlled synthetic docs). Version changes, conflicting docs, stale docs, query traffic patterns, reviewer resolutions, incident/failure events, migration scenarios, dependency manifests, and target versions (supplied via dependency_target_registry.json) are all scripted and controlled.
LOGGED	Every query → query_audit_log. Every retrieval trace → retrieval_traces. Every conflict → conflict_alerts. Every fallback → fallback_events. Every agent goal, task, task run, and recovery action → agentic audit schemas.
NOT CLAIMED	Real production traffic. Real user base. Real revenue. Real SLA obligations. Live npm / PyPI / Maven registry integration. Real GitHub PR generation. Regulatory or security approval.

Key principle: a reviewer who clones the repo and runs `seed_devpulse.py` + `seed_devpulse_v3.py` (or a single wrapper script) should see deterministic evidence of every claimed behavior within 15 minutes. If a behavior cannot be demonstrated via a generated artifact, it must not be claimed.

5. System Architecture Overview

DevPulse v3.0 operates in two modes sharing all underlying infrastructure. Query Mode handles single migration or developer queries. Goal Mode accepts a high-level migration goal and dependency manifest and decomposes it into an ordered set of Query Mode executions with bounded recovery and aggregate reporting.

QUERY MODE	GOAL MODE
Single migration / developer query ↓ QueryParser VersionExtractor ComplexityRouter HybridRetrieval (BM25+pgvector+RRF) ConflictDetector (9 types) MigrationReporter LLMSynthesizer* (conditional) PostGenValidation CitationAssembler ResponseFormatter ↓ Migration Decision Report	Goal text + dependency manifest ↓ GoalParser DependencyDeltaDetector TaskPlanner / TaskQueue ↓ TaskExecutor (per-task loop) ■ calls Query Mode pipeline per task RecoveryDecider (rule-based, cap=2) ↓ PlanSummaryReporter ↓ Aggregate Plan Report (SAFE/RISKY/BLOCKED)

Shared infrastructure (both modes): L1–L11 Query Mode pipeline · BM25+pgvector+RRF · ConflictDetector · MigrationReporter · LLMSynthesizer* · CitationAssembler · query_audit_log · retrieval_traces · conflict_alerts · fallback_events · Langfuse · Sentry

Design Invariants

- TaskExecutor never bypasses any Query Mode layer — version pre-filtering, conflict detection, post-gen validation, and citation assembly all apply at the task level.
- The LLM never plans, routes, detects conflicts, assembles citations, classifies risk, or makes recovery decisions — in either mode.
- Every step in both modes is logged to an auditable schema. No behavior is claimed without a corresponding artifact.
- Retry caps are explicit and non-negotiable: maximum 2 retries per task; $\geq 40\%$ BLOCKED tasks trigger goal-level halt.
- Wrong-version answer rate target = 0.0 — this applies to every query, every task, and every agent run.

6. Query Mode — Version-Aware Migration Decision Core

Query Mode is the version-aware retrieval and migration decisioning foundation. It handles single developer or migration queries end-to-end and provides the correctness infrastructure that Goal Mode task execution builds upon.

L1 Ingestion Pipeline

Type: Async, background

Crawler → Cleaner → Chunker → Embedder → Indexer. Structural chunking preserves change entry boundaries. Metadata enrichment: version_tag, change_type (added/deprecated/removed/modified), api_names[], has_deprecation, replacement_api. Content hash deduplication. Source freshness timestamps.

L2 Document & Chunk Store

Type: PostgreSQL + pgvector

documents(id, source_url, version_tag, doc_type, ingested_at, content_hash, freshness_score). chunks(id, doc_id, content, version_tag, change_type, api_names[], has_deprecation, replacement_api, embedding vector(1536), chunk_hash, created_at).

L3 Query Parser

Type: Deterministic

NLP tokenization, entity extraction (API names, version strings), intent classification (lookup / comparison / migration / compatibility). Normalises version strings to semver. Flags ambiguous version references.

L4 Version Extractor

Type: Deterministic

Extracts explicit versions ('v3.2') and inferred versions ('latest', 'current'). Explicit version → hard pre-filter on version_tag. Inferred version → soft RRF score boost. No version in query → results span versions, conflict check mandatory.

L5 Complexity Router

Type: Deterministic

Routes to: (1) SIMPLE — exact metadata match, BM25 only, no LLM. (2) HYBRID — BM25 + pgvector + RRF, conflict check, conditional LLM. (3) MIGRATION — full pipeline + migration decision report + synthesis required. Routing decision logged to query_audit_log.

L6 Retrieval Engine

Type: Hybrid

BM25 via PostgreSQL tsvector (lexical, strong on API names/version strings). pgvector cosine similarity (semantic, handles paraphrased queries). RRF fusion: $\text{score} = \sum (1 / (k + \text{rank}_i))$, $k=60$. Version pre-filter applied before scoring. Top- $k=10$ candidates, re-ranked to top-5.

L7 Conflict Detector

Type: Deterministic

9 conflict types: same_api_different_behavior · version_deprecation_conflict · changelog_doc_disagreement · replacement_api_ambiguity · breaking_change_missed · additive_change_overstated · parameter_signature_mismatch · stale_doc_detected · cross_source_contradiction. Severity: CRITICAL / HIGH / MEDIUM / LOW. Output: conflict_alert record + conflict_flag on response.

L8 LLM Synthesizer

Type: Conditional

Called only on HYBRID or MIGRATION paths when conflict_flag=False or conflict resolved AND verdict ≠ BLOCKED. Prompt includes retrieved chunks with version tags and explicit instruction to cite only provided sources. Post-generation validation checks all API names against retrieved chunks — ungrounded names → fallback_event logged, synthesis discarded.

L9 Citation Assembler

Type: Deterministic

Programmatic, not LLM-generated. Citations built from chunk metadata: source_url, version_tag, doc_type, section_title. Every claim maps to ≥ 1 citation. Citation confidence = retrieval score.

L10 Migration Decision Report

Type: Deterministic + conditional synthesis

Verdict: SAFE (no conflicts, version coverage complete, grounding_rate=1.0), RISKY (minor conflicts or coverage gaps, synthesis with caveats), BLOCKED (CRITICAL/HIGH unresolved conflict, missing coverage, or grounding_rate < 0.90 — synthesis suppressed, evidence-only response returned).

L11 Observability & Audit

Type: Async logging

query_audit_log · retrieval_traces · conflict_alerts · fallback_events. Langfuse: LLM span traces with token counts and latency. Sentry: error alerts. source_freshness: per-document freshness scores.

LLM-last invariant: The LLM is invoked only at L8, only on HYBRID/MIGRATION paths, only when conflict_flag=False or conflict resolved, and only when verdict ≠ BLOCKED. All upstream layers are deterministic. Post-generation validation at L8 provides a final grounding gate before any LLM output reaches the response.

7. Goal Mode — Agentic Orchestration Layer

Goal Mode accepts a high-level migration goal and a dependency manifest, decomposes the goal into an ordered task queue, runs each task through the Query Mode pipeline, applies deterministic recovery policies for blocked outcomes, and produces an aggregate migration plan. The agentic layer is bounded: it orchestrates the correctness pipeline — it does not replace it.

TaskExecutor does not bypass any Query Mode layer. For every task it executes: version pre-filtering is enforced · conflict detection runs deterministically · citations are assembled programmatically · post-generation validation checks LLM output · every step is logged to both the Query Mode audit schemas and the Goal Mode agentic schemas.

Component 1: GoalParser

Accepts a high-level migration goal and a dependency manifest. Produces a structured, deterministic goal record.

Inputs

- Goal text (e.g., 'Assess migration safety for this repo from SDK v2 to SDK v3')
- **Must-have manifest formats:** package.json, requirements.txt, pasted manifest text
- **Optional manifest formats:** pyproject.toml, pom.xml
- Optional: explicit from_version, to_version, target dependency family, target ecosystem

Processing Rules

- Dependency names and versions extracted deterministically from the manifest file.
- The LLM must not invent dependency versions. Unresolvable versions → version_status: unresolvable.
- Target versions from: (a) explicit user input, (b) dependency_target_registry.json (controlled demo), or (c) flagged as requires_evidence_lookup.
- Ambiguity flags set when: target version missing, manifest partial, multiple version ranges exist.

Component 2: DependencyDeltaDetector

Compares current dependency versions against target versions. Classifies migration risk per dependency deterministically.

dependency_target_registry.json: Controlled demo config file storing target versions for demo dependencies. Prevents hallucinated target versions. Avoids live npm/PyPI registry calls. target_source field is always 'controlled_demo_registry' — inspectable in EA-26.

Risk Classification (deterministic)

Delta Type	Breaking Change Risk
Major version jump	high
Minor with changelog breaking evidence	high
Minor without breaking evidence	medium
Patch	low

Delta Type	Breaking Change Risk
Target version missing or ambiguous	unknown — requires_migration_check: true

Component 3: TaskPlanner / TaskQueue

Converts the parsed goal into a deterministic, priority-ordered task list.

Priority	Condition
1 (highest)	Major version jumps
2	Dependencies with breaking_change_risk = high
3	User-tagged critical runtime path dependencies
4	Dependencies with missing/ambiguous target version
5	Minor version jumps
6 (lowest)	Patch version updates

Within the same priority tier, tasks are ordered alphabetically by dependency name for determinism.

Component 4: TaskExecutor

Runs each task through the full Query Mode pipeline in task-priority order. This is the central design constraint: the agent does not reason freely. It orchestrates the correctness-preserving pipeline across multiple tasks.

1. Build migration query from task fields
2. Run QueryParser (L3)
3. Run VersionExtractor (L4)
4. Run ComplexityRouter → MIGRATION path (L5)
5. Run HybridRetrieval with version pre-filter (L6)
6. Run ConflictDetector — 9 conflict types (L7)
7. Run MigrationReporter → per-task SAFE/RISKY/BLOCKED verdict (L10)
8. [Conditional] Run LLMSynthesizer only if: conflict_flag=False AND verdict≠BLOCKED AND LLM-last rule satisfied (L8)
9. Run PostGenValidation (L8)
10. Run CitationAssembler (L9)
11. Log task run to agent_task_runs · Update task_status in agent_tasks
12. If BLOCKED or ambiguous → hand off to RecoveryDecider

Component 5: RecoveryDecider

Handles task failure states deterministically. All recovery decisions are rule-based. The LLM makes no recovery decisions.

Condition	Recovery Action
BLOCKED — missing to_version coverage	retry_with_cross_source_expansion (once)
BLOCKED — HIGH/CRITICAL conflict	skip_and_escalate — do not synthesise
BLOCKED — stale doc on critical path	retry_with_migration_guide_priority (once)
BLOCKED — low evidence	retry_with_related_version_evidence (once)
Vector store unavailable	fallback_to_bm25_only — log degraded retrieval warning
Retry cap reached (count = 2)	mark_escalated — stop retrying, capped_by_policy = true
LLM synthesis failed	Evidence-only verdict; agent run continues
LLM provider timeout	Provider fallback chain per F-06

Recovery version-correctness rule: Recovery may expand source types, related version evidence, or migration-guide coverage — but it must never allow wrong-version evidence to produce a SAFE verdict. Exact requested-version coverage is still required for SAFE. `retry_with_related_version_evidence` expands evidence scope; it does not lower the version-correctness bar.

Retry Caps (non-negotiable)

- Maximum **2 retries per task** before marking escalated.
- If **≥ 40% of tasks are BLOCKED** → aggregate verdict = BLOCKED regardless of remaining tasks.
- If **any critical-priority (priority 1) task is BLOCKED** → aggregate verdict cannot be SAFE.

Component 6: PlanSummaryReporter

Creates the final goal-level artifact after all tasks have completed, been escalated, or skipped.

Condition	Aggregate Verdict
All tasks SAFE, no escalations	SAFE
Any task RISKY, none BLOCKED or escalated	RISKY
Any critical-priority task BLOCKED or escalated	BLOCKED
≥ 40% of tasks BLOCKED	BLOCKED
Any task escalated regardless of verdict	RISKY at minimum
Degraded retrieval warning present	RISKY at minimum

Staged Migration Recommendation (non-negotiable)

- SAFE tasks can proceed independently.
- RISKY tasks can proceed only with caveats and reviewer approval.
- BLOCKED critical-priority tasks prevent full migration.
- Aggregate verdict cannot be SAFE if any critical task is BLOCKED or escalated.

8. Data Model and Schemas

DevPulse v3.0 uses two groups of schemas. The Query Mode schemas capture all per-query retrieval, conflict, synthesis, and fallback events. The Goal Mode schemas capture goal-level planning, task execution, recovery, and aggregate reporting. The two groups are linked: `agent_task_runs.query_id` references `query_audit_log.query_id`, ensuring every agent task inherits full Query Mode auditability.

Query Mode Schemas

Schema: documents

Field	Type / Description
id	UUID PRIMARY KEY
source_url	TEXT NOT NULL
version_tag	VARCHAR(32)
doc_type	VARCHAR(32) — reference / changelog / migration_guide
ingested_at	TIMESTAMPTZ DEFAULT NOW()
content_hash	VARCHAR(64) — deduplication key
freshness_score	FLOAT — 1.0=current, 0.0=confirmed stale

Schema: chunks

Field	Type / Description
id	UUID PRIMARY KEY
doc_id	UUID REFERENCES documents(id)
content	TEXT NOT NULL
version_tag	VARCHAR(32)
change_type	VARCHAR(32) — added/deprecated/removed/modified
api_names	TEXT[] — extracted API surface references
has_deprecation	BOOLEAN DEFAULT FALSE
replacement_api	TEXT
embedding	vector(1536)
chunk_hash	VARCHAR(64)
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: query_audit_log

Field	Type / Description
query_id	UUID PRIMARY KEY

Field	Type / Description
raw_query	TEXT NOT NULL
parsed_intent	VARCHAR(32)
version_extracted	VARCHAR(32)
route_taken	VARCHAR(16) — SIMPLE / HYBRID / MIGRATION
retrieval_latency_ms	INTEGER
synthesis_latency_ms	INTEGER
conflict_flag	BOOLEAN DEFAULT FALSE
verdict	VARCHAR(16) — SAFE / RISKY / BLOCKED
llm_tokens_used	INTEGER
total_cost_usd	NUMERIC(8,6)
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: retrieval_traces

Field	Type / Description
trace_id	UUID PRIMARY KEY
query_id	UUID REFERENCES query_audit_log(query_id)
chunk_id	UUID REFERENCES chunks(id)
bm25_score	FLOAT
vector_score	FLOAT
rrf_score	FLOAT
rank	INTEGER
selected	BOOLEAN
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: conflict_alerts

Field	Type / Description
id	UUID PRIMARY KEY
query_id	UUID REFERENCES query_audit_log(query_id)
conflict_type	VARCHAR(64) — one of 9 types
severity	VARCHAR(16) — CRITICAL / HIGH / MEDIUM / LOW
chunk_ids	UUID[] — conflicting chunks
description	TEXT NOT NULL

Field	Type / Description
auto_resolved	BOOLEAN DEFAULT FALSE
resolved_by	VARCHAR(64) — system / reviewer
resolved_at	TIMESTAMPTZ
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: fallback_events

Field	Type / Description
event_id	UUID PRIMARY KEY
query_id	UUID REFERENCES query_audit_log(query_id)
reason	VARCHAR(64) — ungrounded_api / vector_store_unavailable / llm_timeout / stale_doc / synthesis_failed
fallback_path	VARCHAR(64)
outcome	VARCHAR(32)
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: source_freshness

Field	Type / Description
doc_id	UUID REFERENCES documents(id)
freshness_score	FLOAT — 1.0=current, 0.0=confirmed stale
last_checked_at	TIMESTAMPTZ
last_updated_at	TIMESTAMPTZ
stale_flag	BOOLEAN DEFAULT FALSE

Goal Mode Schemas

Schema: agent_goals

Field	Type / Description
goal_id	UUID PRIMARY KEY
raw_goal_text	TEXT NOT NULL
manifest_type	VARCHAR(32) — package.json / requirements.txt / pyproject.toml / pom.xml / text
project_name	VARCHAR(128)
ecosystem	VARCHAR(32) — npm / pip / maven / unknown
from_version_context / to_version_context	VARCHAR(64)

Field	Type / Description
parsed_status	VARCHAR(16) — complete / partial / ambiguous / failed
ambiguity_flag	BOOLEAN DEFAULT FALSE
ambiguity_reason	TEXT
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: agent_tasks

Field	Type / Description
task_id	UUID PRIMARY KEY
goal_id	UUID REFERENCES agent_goals(goal_id)
dependency_name	VARCHAR(128) NOT NULL
current_version / target_version	VARCHAR(64)
version_delta_type	VARCHAR(16) — patch / minor / major / unknown
breaking_change_risk	VARCHAR(16) — low / medium / high / unknown
priority	INTEGER NOT NULL
reason_for_priority	TEXT NOT NULL
task_status	VARCHAR(16) — pending / running / safe / risky / blocked / skipped / escalated
created_at / completed_at	TIMESTAMPTZ

Schema: agent_task_runs

Field	Type / Description
run_id	UUID PRIMARY KEY
task_id	UUID REFERENCES agent_tasks(task_id)
query_id	UUID REFERENCES query_audit_log(query_id) ← links to Query Mode audit
route_taken	VARCHAR(16) — SIMPLE / HYBRID / MIGRATION
retrieval_status	VARCHAR(16) — success / degraded / failed
conflict_flag	BOOLEAN DEFAULT FALSE
migration_verdict	VARCHAR(16) — SAFE / RISKY / BLOCKED
grounding_rate	NUMERIC(4,3)
retry_count	INTEGER DEFAULT 0
final_status	VARCHAR(16) — safe / risky / blocked / escalated / skipped
latency_ms / cost_usd	INTEGER / NUMERIC(8,6)

Field	Type / Description
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: recovery_actions

Field	Type / Description
recovery_id	UUID PRIMARY KEY
task_id	UUID REFERENCES agent_tasks(task_id)
run_id	UUID REFERENCES agent_task_runs(run_id)
recovery_action	VARCHAR(64) — retry_with_cross_source_expansion / skip_and_escalate / etc.
reason	TEXT NOT NULL — human-readable trigger condition
retry_number	INTEGER NOT NULL — 1 or 2
outcome	VARCHAR(16) — success / still_blocked / escalated
capped_by_policy	BOOLEAN DEFAULT FALSE
created_at	TIMESTAMPTZ DEFAULT NOW()

Schema: plan_summary_reports

Field	Type / Description
report_id	UUID PRIMARY KEY
goal_id	UUID REFERENCES agent_goals(goal_id)
total_tasks / safe_tasks / risky_tasks / blocked_tasks	INTEGER
skipped_tasks / escalated_tasks	INTEGER DEFAULT 0
aggregate_verdict	VARCHAR(16) — SAFE / RISKY / BLOCKED
recommended_action	VARCHAR(64) — Proceed / Proceed with caution / Do not proceed
report_path	TEXT — path to plan_summary_report.json
generated_at	TIMESTAMPTZ DEFAULT NOW()

Schema linkage: agent_task_runs.query_id references query_audit_log.query_id. Every Goal Mode task execution inherits full Query Mode retrieval traceability via retrieval_traces and full conflict auditability via conflict_alerts.

9. Retrieval, Version Correctness, and Conflict Detection

Version Correctness — The Core Constraint

Version correctness is the primary correctness property of DevPulse. A system that retrieves the right answer for the wrong version is worse than no system — it generates confident incorrect guidance. All retrieval design flows from this constraint.

Query Type	Version Handling	Retrieval Behaviour
Explicit version ('v3.2 authenticate')	Hard pre-filter: WHERE version_tag='3.2'	Only v3.2 chunks eligible. Zero tolerance for wrong-version chunks.
Inferred version ('latest authenticate')	Soft boost: RRF score boost for most_recent_version chunks	Recency-boosted but not hard-filtered. Conflict check required.
No version ('how does authenticate work')	No filter. All versions eligible.	Conflict check mandatory. Migration decision report if versions diverge.
Version range ('v2 to v3 migration')	Multi-version retrieval: fetch both version sets	Diff-style result. Conflict detection across both version sets.

Retrieval Design Principles

Signal Type	Mechanism	Why
Exact API name match	BM25 tsvector	Lexical matching outperforms embeddings on exact technical strings
Version filtering	SQL WHERE + metadata pre-filter	Hard correctness constraint — cannot be approximate
Semantic paraphrase	pgvector cosine similarity	Handles natural language variations around the same concept
Score fusion	Reciprocal Rank Fusion (k=60)	Combines BM25 and vector ranks without normalisation complexity
Recency boost	RRF score adjustment	Most-recent-version chunks boosted for 'latest' version queries
Change type filter	metadata filter (change_type IN (...))	Migration queries prioritise deprecated/removed/modified chunks

9 Conflict Types with Severity

Conflict Type	Severity	Description
same_api_different_behavior	CRITICAL	Same API described differently in two retrieved chunks from different versions
version_deprecation_conflict	HIGH	Chunk A says deprecated in v3; Chunk B (v3 ref doc) still shows it as current
changelog_doc_disagreement	HIGH	Changelog says removed in v3.1; reference doc present without deprecation marker
replacement_api_ambiguity	HIGH	Two sources disagree on replacement API name or signature

Conflict Type	Severity	Description
breaking_change_missed	CRITICAL	Retrieved chunks do not surface a known breaking change between requested versions
additive_change_overstated	MEDIUM	Changelog describes change as additive; doc shows behavior change breaking for some
parameter_signature_mismatch	HIGH	Same method described with different parameter signatures across chunks
stale_doc_detected	MEDIUM	Chunk freshness_score below threshold; may not reflect current behaviour
cross_source_contradiction	HIGH	Two trusted sources (official docs vs. migration guide) contradict each other

10. Migration Decision Report Design

The migration decision report is the highest-value output of DevPulse Query Mode. It is produced for all MIGRATION path queries and gives a structured verdict that a platform engineer can act on: SAFE, RISKY, or BLOCKED.

Verdict	Conditions	System Behaviour
SAFE	No conflicts. All sources agree. Version coverage complete (both from- and to-versions). No stale docs. Grounding rate = 1.0.	Synthesis proceeds. Full migration summary produced. Citations include version tags for all claims.
RISKY	Minor conflicts (MEDIUM severity only) or coverage gaps in one doc type. Stale doc present but not on critical path. Grounding rate >= 0.90.	Synthesis proceeds with explicit caveats. Conflict summary included. Reviewer resolution recommended before rollout.
BLOCKED	Any CRITICAL or HIGH unresolved conflict. Missing to-version coverage. Stale doc on critical path. Grounding rate < 0.90. Post-gen validation failed.	Synthesis suppressed. Evidence-only response returned with citations. Explicit: 'Insufficient evidence for safe migration recommendation.' Reviewer resolution required.

Migration Decision Report — Output Fields

Field	Description
verdict	SAFE / RISKY / BLOCKED
from_version / to_version	Versions being compared
query_id	Audit log reference
conflict_count	{critical: N, high: N, medium: N, low: N}
version_coverage	{from_version: complete/partial/missing, to_version: complete/partial/missing}
source_freshness_min	Minimum freshness_score across all retrieved chunks

Field	Description
grounding_rate	Fraction of synthesis output claims grounded in retrieved chunks
citations	[{source_url, version_tag, doc_type, section_title, retrieval_score}]
synthesis_text	LLM-generated migration summary (null if BLOCKED)
caveats	List of explicit caveats: conflicts, gaps, stale docs
generated_at	Timestamp

11. Observability and Audit Design

Observability is designed not only for uptime and latency, but for preventing wrong-version or stale-guidance failures. Every decision point in both modes is logged to an auditable store.

Operational Metrics — Query Mode

Metric	Definition	Target Gate	Priority
version_error_rate	Wrong-version chunks returned / total chunks returned	= 0.0	CRITICAL
conflict_detection_rate	Conflicts detected / conflicts injected in eval	>= 0.90	HIGH
synthesis_grounding_rate	Grounded synthesis outputs / total synthesis calls	>= 0.95	HIGH
stale_doc_fallback_rate	Stale-doc fallbacks fired / stale docs in corpus	= 1.0	HIGH
p95_hybrid_latency_ms	95th pct latency for HYBRID path	<= 800ms	MEDIUM
p95_migration_latency_ms	95th pct latency for MIGRATION path	<= 2000ms	MEDIUM
llm_cost_per_query_usd	Average LLM cost across all paths	<= \$0.003	LOW
blocked_verdict_rate	BLOCKED verdicts / total migration queries	Alert if > 0.3	MEDIUM

Operational Metrics — Goal Mode (additional)

Metric	Definition	Target Gate
unbounded_retry_rate	Tasks with retry_count > 2 / total tasks	= 0.0
llm_planning_violation_rate	Tasks where LLM used for planning / routing / recovery	= 0.0
aggregate_verdict_correctness	Correct aggregate verdicts / total goal runs	= 1.0
recovery_policy_correctness	Recovery actions matching policy rules / total recovery events	= 1.0

Metric	Definition	Target Gate
trace_completeness_rate	Task runs with all required fields populated / total runs	>= 0.98

All numeric values above are target acceptance gates — not claimed implementation results. They become verifiable only after the corresponding evidence artifacts (EA-15, EA-16, EA-31) exist.

12. LLM-Last Orchestration Rules

Strict LLM-Last Rule (non-negotiable): The LLM must never own version filtering, conflict detection, citation assembly, source freshness checks, fallback decisions, recovery policy, task planning, goal decomposition, or risk classification. The LLM is permitted only to synthesise already-retrieved, already-validated evidence. Any system design that delegates these responsibilities to a model violates the core correctness guarantee of DevPulse.

Query Mode Execution Flow

#	Node	Type	Logic
1	QueryParser	Deterministic	Tokenise, extract entities, classify intent → ParsedQuery
2	VersionExtractor	Deterministic	Extract version hints → explicit / inferred / none → VersionContext
3	ComplexityRouter	Deterministic	Route to SIMPLE / HYBRID / MIGRATION based on intent + version context
4a	SimpleRetrieval	BM25 + metadata filter	No vector search, no LLM. Return top-3 chunks with citations
4b	HybridRetrieval	BM25 + pgvector + RRF	Version pre-filter, RRF fusion, top-10 → top-5
5	ConflictDetector	Deterministic	Check 9 conflict types → conflict_alerts[], conflict_flag
6	MigrationReporter	Deterministic	Assess version coverage → MigrationDecisionReport
7	LLMSynthesizer	LLM (conditional)	Only if: HYBRID/MIGRATION AND conflict_flag=False AND verdict≠BLOCKED
8	PostGenValidation	Deterministic	Check all API names against retrieved chunks → fallback_event if ungrounded
9	CitationAssembler	Deterministic	Build programmatic citations from chunk metadata
10	ResponseFormatter	Deterministic	Format response with verdict badge, citations, conflict warning

LLM Prompt Structure

```
SYSTEM: You are a version-aware API documentation assistant.  
Use ONLY the provided source chunks. Do not use external knowledge.  
Cite every claim with [chunk_id: X, version: Y].  
If evidence is insufficient or conflicting, say so explicitly.  
  
USER: Query: {parsed_query}  
Target version: {version_context}  
Source chunks: [chunk_id: 1, version: v3.2, source: reference_docs] {chunk_content} ...  
Conflict context (if any): {conflict_summary}
```

Goal Mode Determinism Guarantees

Operation	Owner	LLM Involved?
Manifest parsing and dependency extraction	GoalParser (deterministic)	Never
Target version resolution	dependency_target_registry.json + explicit input	Never
Ambiguity detection	GoalParser (rule-based)	Never
Version delta classification	DependencyDeltaDetector (semver rules)	Never
Task priority ordering	TaskPlanner (priority table)	Never
Per-task retrieval and conflict detection	Query Mode L3–L7	Never for retrieval/detection
Migration verdict per task	MigrationReporter (deterministic)	Never
Recovery decision	RecoveryDecider (rule table)	Never
Aggregate verdict	PlanSummaryReporter (rule logic)	Never
Synthesis of already-validated evidence	LLMSynthesizer (L8)	Yes — bounded, conditional only

13. 37-Day Simulated Operating Lifecycle

The full operating lifecycle is generated by two scripts: `seed_devpulse.py` (Days 1–30, Query Mode core) and `seed_devpulse_v3.py` (Days 31–37, Goal Mode agentic extension). A wrapper script may run both in sequence. Every event is timestamped, logged, and verifiable. The lifecycle exercises every system path at least once.

Days 1–30: Query Mode Core Lifecycle

Day 1–3: Ingestion & Indexing

- Ingest 3 API doc sources: reference docs, changelog, migration guide
- Build BM25 index (PostgreSQL tsvector) and pgvector index from ingested chunks
- Assert metadata fill rate: version_tag and change_type populated for $\geq 95\%$ of chunks
- Log baseline source freshness scores

Evidence artifact: EA-01 ingest_summary.json

Exit gate: Metadata fill rate $\geq 95\%$; version_tag populated for all ingested chunks

Day 4–7: Simple Query Baseline

- Run 20 simple lookup queries via SIMPLE path (no LLM called)
- Assert $P@1 \geq 0.90$ for exact API name queries
- Assert all responses have ≥ 1 citation with correct version_tag
- Log all queries to query_audit_log

Evidence artifact: EA-06 simple_query_results.json

Exit gate: $P@1 \geq 0.90$ on exact API name queries; zero wrong-version chunks returned

Day 8–12: Hybrid Retrieval Validation

- Run 30 hybrid queries: version comparisons, deprecation lookups, semantic queries
- Assert RRF fusion improves $P@5$ vs BM25-only by $\geq 10\%$
- Assert version filtering: no v2 chunks in responses to explicit v3 queries
- Log retrieval_traces for all queries

Evidence artifact: EA-07 hybrid_retrieval_report.json · EA-08 version_filter_audit.json · EA-09 retrieval_traces_sample.json

Exit gate: RRF improvement $\geq 10\%$; zero wrong-version chunks in EA-08

Day 13–16: Conflict Detection

- Inject 9 conflict scenarios — one per conflict type
- Assert conflict_alerts created for all 9 scenarios
- Assert BLOCKED verdict on CRITICAL severity conflicts
- Assert conflict_flag=True on responses with unresolved HIGH/CRITICAL conflicts

Evidence artifact: EA-10 conflict_detection_report.json · EA-11 conflict_alerts_schema.sql

Exit gate: All 9 conflict types fire on injected scenarios; BLOCKED verdict on CRITICAL

Day 17–20: LLM Synthesis Path

- Run 15 migration queries requiring synthesis
- Assert all LLM responses cite only retrieved chunks (post-generation validation)
- Inject 3 ungrounded API names — assert fallback_event logged and synthesis discarded
- Assert migration decision reports produced for all MIGRATION path queries

Evidence artifact: EA-12 migration_decision_samples.json · EA-13 synthesis_grounding_report.json · EA-14 citation_assembly_sample.json

Exit gate: *Grounding rate ≥ 0.95 ; fallback_events logged for all 3 ungrounded injections*

Day 21–25: Evaluation Harness

- Run full golden eval suite: 50+ queries across 10 categories
- Assert version conflict detection accuracy ≥ 0.90
- Assert wrong-version answer rate = 0.0 (target acceptance gate)
- Run adversarial wrong-version traps: assert BLOCKED or conflict flag, not wrong-version answer

Evidence artifact: EA-15 golden_eval_results.json · EA-16 adversarial_trap_results.json

Exit gate: *Version accuracy = 1.0; wrong-version answer rate = 0.0; conflict detection rate ≥ 0.90*

Day 26–28: Stale Doc & Freshness

- Simulate doc staleness: mark 2 chunks with freshness_score below threshold
- Assert stale-doc fallback fires: response includes staleness warning
- Re-ingest updated docs; assert freshness scores updated
- Assert embedding replacement completes without query downtime (blue/green swap)

Evidence artifact: EA-19 freshness_report.json · EA-20 embedding_swap_log.txt

Exit gate: *Stale-doc fallback rate = 1.0; embedding swap verified without downtime*

Day 29–30: Full Query Mode Demo

- Run show_demo_report.py to generate terminal evidence summary
- Assert all 24 Query Mode evidence artifacts present and non-empty
- Assert all golden eval thresholds met
- Assert no CRITICAL unresolved conflicts in audit log

Evidence artifact: EA-17 query_audit_log_sample.json · EA-21 langfuse_trace_export.json · EA-22 sentry_error_summary.txt · EA-23 cost_latency_report.json · EA-24 devpulse_demo_report.txt

Exit gate: *show_demo_report.py runs end-to-end; all 24 artifacts present; all thresholds met*

Days 31–37: Goal Mode Agentic Extension Lifecycle

Days 31–37 require the Query Mode lifecycle (Days 1–30) to have completed first. seed_devpulse_v3.py generates this extension.

Day 31: GoalParser + Manifest Parsing

- Author dependency_target_registry.json with controlled demo target versions
- Ingest sample dependency manifest (5–7 deps: major, minor, patch mix)
- Run GoalParser over manifest + goal text
- Assert parsed_status = complete for well-formed input; assert ambiguity_flag = true when target version missing
- Assert no invented versions in dependency_list

Evidence artifact: EA-25 agent_goal_parse_sample.json

Exit gate: *parsed_status = complete AND no unresolvable versions marked as resolved*

Day 32: DependencyDeltaDetector + Controlled Registry

- Run DependencyDeltaDetector over parsed goal (reads from dependency_target_registry.json)
- Assert version_delta_type correctly classified for all resolvable deps
- Assert target_source = 'controlled_demo_registry' in all resolved records
- Assert breaking_change_risk = unknown for missing target versions
- Assert at least one dep classified as major / high risk

Evidence artifact: EA-26 dependency_delta_report.json · dependency_target_registry.json (supporting config)

Exit gate: *version_delta_type populated for all resolvable deps; no live registry calls; risk classification matches semver rules*

Day 33: TaskPlanner / TaskQueue

- Run TaskPlanner over EA-26 output
- Assert tasks ordered by priority (major version jumps first)
- Assert reason_for_priority populated for all tasks
- Assert at least one ambiguous task placed correctly in priority order

Evidence artifact: EA-27 agent_task_plan.json

Exit gate: *Task count matches dependency count; priority ordering deterministic; all task_status = pending*

Day 34: TaskExecutor Loop

- Run TaskExecutor over the task queue from Day 33
- Assert query_id populated in agent_task_runs (links to query_audit_log)
- Assert at least one task SAFE, one RISKY, one BLOCKED
- Assert retrieval_traces and conflict_alerts populated for tasks that triggered them
- Assert synthesis suppressed for BLOCKED tasks

Evidence artifact: EA-28 agent_task_execution_trace.json

Exit gate: *All tasks have final_status; query_id populated per task; no task has retry_count > 0 at this stage*

Day 35: RecoveryDecider — BLOCKED Scenarios

- Inject BLOCKED-due-to-missing-docs: assert retry_with_cross_source_expansion fires
- Inject BLOCKED-due-to-HIGH-conflict: assert skip_and_escalate (no retry)
- Inject low-evidence BLOCKED: assert retry_with_related_version_evidence fires; task moves BLOCKED → RISKY (not SAFE)
- Inject retry-cap scenario: assert capped_by_policy = true after retry 2
- Assert unbounded_retry_rate = 0 (no task has retry_count > 2)

Evidence artifact: EA-29 recovery_decision_log.json

Exit gate: All recovery actions logged; at least one BLOCKED→RISKY improvement (F-19); capped_by_policy=true for cap-hit tasks

Day 36: PlanSummaryReporter

- Run PlanSummaryReporter over all task outcomes from Days 34–35
- Assert aggregate_verdict logic correctly applied (F-16 scenario covered)
- Assert do_not_proceed_blockers lists all escalated / BLOCKED critical-priority tasks
- Assert staged migration recommendation present: SAFE tasks listed as proceed-able independently

Evidence artifact: EA-30 plan_summary_report.json

Exit gate: aggregate_verdict non-null; staged recommendation present; do_not_proceed_blockers populated when critical task BLOCKED

Day 37: Agentic Demo + Evaluation

- Run full agentic eval harness over Days 31–36 outputs
- Assert all agentic eval metrics meet target gates
- Assert unbounded_retry_rate = 0; llm_planning_violation_rate = 0
- Run show_agentic_demo.py end-to-end; capture terminal output

Evidence artifact: EA-31 agentic_eval_results.json · EA-32 agentic_demo_report.txt

Exit gate: show_agentic_demo.py runs in under 3 minutes; all agentic eval thresholds met; all 8 Goal Mode artifacts present

14. Evidence Artifacts — EA-01 to EA-32

Every artifact is real output from the running system. Format: content · how produced · interview signal. All 32 artifacts must be present and non-empty before the final demo. EA-01 to EA-24 are produced by seed_devpulse.py (Days 1–30). EA-25 to EA-32 are produced by seed_devpulse_v3.py (Days 31–37).

Query Mode Artifacts — EA-01 to EA-24**EA-01: ingest_summary.json**

Contents: Chunk count, version coverage, metadata fill rate per source

How produced: Ingestion pipeline run on Day 1–3

Interview signal: Ingestion pipeline quality and metadata enrichment correctness

EA-02: chunk_metadata_sample.json

Contents: 20 representative chunks with all metadata fields populated

How produced: Sampled during Day 1–3 ingestion

Interview signal: Metadata enrichment correctness; version_tag and change_type populated

EA-03: bm25_index_stats.txt

Contents: Term count, document frequency distribution

How produced: BM25 index build on Day 1–3

Interview signal: Lexical index health and term coverage

EA-04: pgvector_index_stats.txt

Contents: Vector count, dimensionality, index type, build time

How produced: pgvector index build on Day 1–3

Interview signal: Vector index setup and embedding dimension verification

EA-05: version_coverage_matrix.json

Contents: Which versions have coverage in which doc types

How produced: Generated from documents table on Day 3

Interview signal: Version-aware indexing completeness

EA-06: simple_query_results.json

Contents: 20 simple lookup results with scores and citations

How produced: SIMPLE path queries on Days 4–7

Interview signal: BM25 + metadata filter path performance; citation correctness

EA-07: hybrid_retrieval_report.json

Contents: BM25 vs hybrid P@5 comparison across 30 queries

How produced: Hybrid retrieval runs on Days 8–12

Interview signal: RRF fusion effectiveness vs BM25-only baseline

EA-08: version_filter_audit.json

Contents: Proof that explicit version queries return zero wrong-version chunks

How produced: Version filter assertion on Days 8–12

Interview signal: Version correctness enforcement — the key correctness artifact

EA-09: retrieval_traces_sample.json

Contents: 10 full retrieval traces with bm25_score, vector_score, rrf_score, rank

How produced: Sampled from retrieval_traces on Days 8–12

Interview signal: Retrieval transparency and RRF scoring visibility

EA-10: conflict_detection_report.json

Contents: All 9 conflict types detected, severity assigned, alerts created

How produced: 9 injected conflict scenarios on Days 13–16

Interview signal: Conflict detection coverage across all 9 types

EA-11: conflict_alerts_schema.sql

Contents: Full DDL for conflict_alerts table

How produced: Generated from implemented schema on Day 13

Interview signal: Schema design; conflict taxonomy implementation

EA-12: migration_decision_samples.json

Contents: 5 migration decisions with SAFE/RISKY/BLOCKED verdicts and evidence summaries

How produced: MIGRATION path queries on Days 17–20

Interview signal: Migration decision logic and verdict conditions

EA-13: synthesis_grounding_report.json

Contents: Grounding rate across 15 synthesis queries; fallback events for ungrounded outputs

How produced: Synthesis path runs on Days 17–20

Interview signal: LLM bounding via post-generation validation

EA-14: citation_assembly_sample.json

Contents: 10 responses with programmatic citations and chunk source mapping

How produced: Citation assembler output on Days 17–20

Interview signal: Programmatic citation correctness

EA-15: golden_eval_results.json

Contents: Full eval suite: per-category metrics, threshold pass/fail across 10 categories

How produced: Golden eval harness run on Days 21–25

Interview signal: Evaluation rigor; version accuracy and wrong-version answer rate

EA-16: adversarial_trap_results.json

Contents: Results for 15 wrong-version trap queries; all must return BLOCKED or conflict flag

How produced: Adversarial eval run on Days 21–25

Interview signal: Version correctness under adversarial conditions

EA-17: query_audit_log_sample.json

Contents: 30-day query log sample with route, latency, verdict per query

How produced: Sampled from query_audit_log on Day 29

Interview signal: Audit trail and routing distribution

EA-18: fallback_events_log.json

Contents: All fallback events: reason, path taken, outcome

How produced: Sampled from fallback_events on Day 29

Interview signal: Fallback correctness and synthesis suppression coverage

EA-19: freshness_report.json

Contents: Source freshness scores before and after staleness simulation

How produced: Freshness simulation on Days 26–28

Interview signal: Stale-doc detection and freshness tracking

EA-20: embedding_swap_log.txt

Contents: Blue/green index swap steps; zero-downtime verification

How produced: Embedding swap simulation on Days 26–28

Interview signal: Embedding replacement without downtime

EA-21: langfuse_trace_export.json

Contents: Sample LLM spans with token counts, latency, prompt hashes

How produced: Langfuse export on Day 29

Interview signal: LLM observability and cost tracing

EA-22: sentry_error_summary.txt

Contents: Error categories observed during 30-day lifecycle

How produced: Sentry export on Day 29

Interview signal: Error monitoring coverage

EA-23: cost_latency_report.json

Contents: P50/P95 latency per path; LLM cost per query type; total lifecycle cost

How produced: Aggregated from query_audit_log on Day 29

Interview signal: Cost/latency discipline; SIMPLE vs HYBRID vs MIGRATION comparison

EA-24: devpulse_demo_report.txt

Contents: Full terminal demo report: all metrics, all thresholds, summary verdict

How produced: show_demo_report.py on Day 30

Interview signal: Final portfolio demo artifact for Query Mode

Goal Mode Artifacts — EA-25 to EA-32

EA-25: agent_goal_parse_sample.json

Contents: Raw goal text, parsed manifest, full dependency list with versions, delta types, risk classifications, ambiguity flags, parsed_status

How produced: GoalParser on Day 31

Interview signal: Deterministic manifest parsing; ambiguity flags prove system does not invent versions

EA-26: dependency_delta_report.json

Contents: Per-dependency: dependency_name, current_version, target_version, version_delta_type, breaking_change_risk, requires_migration_check, version_status, target_source

How produced: DependencyDeltaDetector on Day 32

Interview signal: Version delta classification without LLM; controlled demo registry proof

EA-27: agent_task_plan.json

Contents: Ordered task queue: task_id, dependency_name, versions, delta type, priority, reason_for_priority, initial task_status

How produced: TaskPlanner on Day 33

Interview signal: Deterministic task ordering; priority reasoning is explicit and inspectable

EA-28: agent_task_execution_trace.json

Contents: Per task: task_id, dependency_name, migration query, query_id (links to query_audit_log), route_taken, retrieval_status, conflict_flag, conflict_types, migration_verdict, grounding_rate, retry_count, final_status, latency_ms, cost_usd

How produced: TaskExecutor on Day 34

Interview signal: Most important Goal Mode artifact. Proves agentic layer orchestrates the correctness pipeline. query_id proves Query Mode audit trail applies per task.

EA-29: recovery_decision_log.json

Contents: All recovery_actions records: recovery_id, task_id, dependency_name, recovery_action, reason, retry_number, outcome, capped_by_policy, created_at. Includes F-19 successful BLOCKED→RISKY recovery.

How produced: RecoveryDecider on Day 35

Interview signal: Deterministic bounded recovery; retry cap enforcement; at least one successful improvement scenario

EA-30: plan_summary_report.json

Contents: Complete PlanSummaryReport: original goal, manifest summary, dependency delta table, ordered task plan, per-task trace, task counts, aggregate_verdict with reasoning, staged migration recommendation, do_not_proceed_blockers, recovery summary, citations, generated_at

How produced: PlanSummaryReporter on Day 36

Interview signal: Primary agentic deliverable. Shows full loop: Goal → Plan → Execute → Verify → Recover → Final Decision

EA-31: agentic_eval_results.json

Contents: Per-metric results: target gate, actual value, pass/fail. Sanity checks: unbounded_retry_rate = 0, llm_planning_violation_rate = 0

How produced: Agentic eval harness on Day 37

Interview signal: Proves agentic layer is evaluated, not just described

EA-32: agentic_demo_report.txt

Contents: Full terminal output of agentic loop: goal → parsed manifest → dependency delta → task plan → per-task execution → recovery decisions → final plan summary → aggregate verdict

How produced: show_agentic_demo.py on Day 37

Interview signal: Walk-through demo artifact; hiring reviewer can run from a fresh clone in under 3 minutes

Supporting Config Artifact

dependency_target_registry.json

Purpose: Stores controlled target versions for demo dependencies. Keeps the Goal Mode demo deterministic. Avoids live external package registry calls. Prevents hallucinated target versions.

How produced: Manually authored as part of seed_devpulse_v3.py setup. Read by DependencyDeltaDetector on Day 32.

Interview signal: Demonstrates production-simulated discipline: the builder explicitly controls what 'target version' means in the demo. target_source = 'controlled_demo_registry' is inspectable in EA-26.

15. Failure and Recovery Scenarios — F-01 to F-19

All 19 scenarios are deterministic and scriptable. Each has: inject step, expected system behaviour, and recovery/verification. F-01 to F-10 exercise Query Mode. F-11 to F-19 exercise Goal Mode. All scenarios must be executable against the 37-day lifecycle artifacts.

Query Mode Scenarios — F-01 to F-10

F-01: Wrong-version chunk returned	
INJECT	Insert chunk with version_tag=v2 into a v3-only query result set
EXPECTED	version_filter_audit.json (EA-08) shows 0 wrong-version chunks in filtered results
RECOVERY	Version pre-filter prevents inclusion; asserted in CI golden eval gate

F-02: LLM hallucinates API name	
INJECT	Craft prompt where LLM likely invents api.nonexistent()
EXPECTED	fallback_event logged; synthesis discarded; response returns BLOCKED with evidence only
RECOVERY	Post-generation validation catches ungrounded API names; synthesis suppressed

F-03: Critical conflict not flagged

INJECT	Inject two chunks: same API, different behaviour, different versions
EXPECTED	conflict_alert created: type=same_api_different_behavior, severity=CRITICAL; verdict=BLOCKED
RECOVERY	ConflictDetector fires deterministically; response returns BLOCKED

F-04: Stale doc trusted

INJECT	Set freshness_score=0.1 on a chunk (below threshold=0.4)
EXPECTED	Response includes staleness warning; fallback_event logged with reason=stale_doc
RECOVERY	Freshness check in retrieval post-filter; stale chunks deprioritised

F-05: pgvector unavailable

INJECT	Kill pgvector connection during a hybrid query
EXPECTED	System falls back to BM25-only path; fallback_event logged with reason=vector_store_unavailable
RECOVERY	Graceful degradation; no crash; latency SLA still met on BM25 path

F-06: LLM provider timeout

INJECT	Simulate Groq timeout (>5s) on a synthesis call
EXPECTED	System falls back to OpenAI; if both fail, synthesis suppressed and deterministic evidence returned
RECOVERY	Provider fallback chain; synthesis suppression as last resort

F-07: Eval regression on version accuracy

INJECT	Commit change that degrades version filtering logic
EXPECTED	CI golden eval gate fails; deployment blocked
RECOVERY	CI regression gate catches degradation before merge

F-08: Embedding model changed without re-index

INJECT	Update embedding model config without triggering re-index
EXPECTED	Startup validation detects embedding_model mismatch; startup aborted
RECOVERY	Config validation at startup; blue/green index ensures old index remains available

F-09: Adversarial version-ambiguous query

INJECT	Query: 'how does authenticate() work?' — no version hint, breaking change between v2 and v3
EXPECTED	Conflict detected (same_api_different_behavior); response returns RISKY verdict with both versions surfaced
RECOVERY	ConflictDetector + migration decision report handle version ambiguity correctly

F-10: Changelog and reference doc disagree

INJECT	Inject changelog saying method removed in v3; reference doc still describes it
EXPECTED	conflict_alert: type=changelog_doc_disagreement, severity=HIGH; synthesis suppressed or heavily caveated
RECOVERY	Cross-source conflict detection fires; reviewer resolution workflow triggered

Goal Mode Scenarios — F-11 to F-19**F-11: Ambiguous Migration Goal**

INJECT	Input goal text contains no target version ('Upgrade all SDKs to latest')
EXPECTED	GoalParser sets ambiguity_flag=true. Affected tasks created with version_status: unresolvable. No invented target version.
RECOVERY	DependencyDeltaDetector marks breaking_change_risk: unknown. TaskExecutor attempts cross-source expansion. If not found, task escalated.

F-12: Unsupported Manifest Format

INJECT	Input manifest is malformed JSON or unrecognised format
EXPECTED	GoalParser sets parsed_status: failed. Agent goal marked BLOCKED immediately. No partial hallucinated plan.
RECOVERY	Error logged to agent_goals. User presented with failure reason and expected format.

F-13: Missing Target-Version Documentation

INJECT	Task has valid target version but no corpus docs exist for that version
EXPECTED	TaskExecutor returns BLOCKED (insufficient_version_coverage). RecoveryDecider applies retry_with_cross_source_expansion (once). Still missing → escalated.
RECOVERY	recovery_actions log shows retry_number=1. If still_blocked, retry_number=2 → skip_and_escalate, capped_by_policy=true.

F-14: High-Severity Conflict During Task Execution

INJECT	ConflictDetector finds CRITICAL conflict (same_api_different_behavior) during task execution
EXPECTED	conflict_alerts record created. Synthesis suppressed. Task verdict=BLOCKED. RecoveryDecider: skip_and_escalate (no retry).
RECOVERY	Task added to do_not_proceed_blockers. Aggregate verdict cannot be SAFE if task is critical-priority.

F-15: Recovery Loop Cap Reached

INJECT	Task is BLOCKED after retry 1 and BLOCKED again after retry 2
EXPECTED	RecoveryDecider enforces cap: capped_by_policy=true. task_status=escalated. No third retry. retry_count=2.
RECOVERY	Task added to do_not_proceed_blockers. agentic_eval_results.json asserts unbounded_retry_rate=0.

F-16: Mixed Aggregate Migration Verdict

INJECT	Goal run: 2 SAFE, 1 RISKY, 1 BLOCKED (low priority), 1 BLOCKED (critical priority=1)
EXPECTED	Because one critical-priority task is BLOCKED: aggregate_verdict=BLOCKED. recommended_action=Do not proceed. Staged recommendation: SAFE tasks could proceed independently.
RECOVERY	Correct aggregate verdict behaviour. EA-30 demonstrates this mixed-verdict scenario.

F-17: Vector Store Unavailable During Agent Execution

INJECT	pgvector killed mid-run after 2 tasks completed, during task 3
EXPECTED	F-05 fires at task level. Task 3 falls back to BM25-only. fallback_event logged. retrieval_status=degraded. Plan report includes degraded_retrieval_warning. Aggregate verdict=RISKY at minimum.
RECOVERY	Remaining tasks continue with BM25-only. Report documents which tasks ran under degraded retrieval.

F-18: LLM Synthesis Fails for One Task

INJECT	LLM provider times out on one task's synthesis call
EXPECTED	F-06 provider fallback fires. If all providers fail, synthesis suppressed. Task receives evidence-only deterministic verdict. grounding_rate=null. Agent run continues.
RECOVERY	Evidence-only verdict is valid. LLM failure does not block the goal run. synthesis_suppressed: true in EA-28.

F-19: Successful Recovery from Missing Evidence

INJECT	Initial retrieval for one task lacks target-version migration-guide coverage. First task run returns BLOCKED (insufficient_evidence_coverage).
EXPECTED	RecoveryDecider applies retry_with_related_version_evidence (retry 1). Second run finds related migration-guide evidence sufficient for a caveated recommendation. Task verdict moves from BLOCKED to RISKY — not SAFE, because exact full target-version coverage is still incomplete.
RECOVERY	EA-29 shows retry action, trigger condition, and improved outcome. EA-28 shows first verdict BLOCKED, second verdict RISKY, retry_count=1. EA-30 includes task as RISKY with caveats. Proves RecoveryDecider can improve task state while respecting version-correctness boundaries.

16. Evaluation Design and Acceptance Gates

All numeric values in this section are target acceptance gates — implementation acceptance thresholds that become verifiable only after the corresponding evidence artifacts exist. They are not claimed as achieved results until EA-15, EA-16, and EA-31 are produced.

Query Mode Golden Eval Suite: 10 Categories, 50+ Queries

Cat	Category	Queries	Target Acceptance Gate	Signal
CAT-1	Exact API lookups	15	P@1 >= 0.95	BM25 + metadata filter strength
CAT-2	Version-specific lookups	10	Version accuracy = 1.0	Hard version filter enforcement
CAT-3	Deprecation queries	8	has_deprecation recall >= 0.95	Metadata enrichment quality
CAT-4	Migration path queries	8	Grounding rate >= 0.90	Synthesis quality on MIGRATION path
CAT-5	Version comparison queries	6	Conflict detection rate >= 0.90	Conflict detection coverage
CAT-6	Semantic paraphrase queries	8	P@5 >= 0.80	Vector retrieval quality

Cat	Category	Queries	Target Acceptance Gate	Signal
CAT-7	Adversarial wrong-version traps	10	Wrong-version answer rate = 0.0	Version correctness under adversarial conditions
CAT-8	Stale doc queries	5	Stale-doc fallback rate = 1.0	Freshness handling
CAT-9	No-confident-answer queries	5	BLOCKED / abstain rate ≥ 0.85	Uncertainty handling
CAT-10	Cross-source conflict queries	6	Conflict flag rate ≥ 0.90	Cross-source conflict detection

Query Mode Operational Metrics

Metric	Definition	Gate	Priority
version_error_rate	Wrong-version chunks returned / total chunks returned	= 0.0	CRITICAL
conflict_detection_rate	Conflicts detected / conflicts injected	≥ 0.90	HIGH
synthesis_grounding_rate	Grounded outputs / total synthesis calls	≥ 0.95	HIGH
stale_doc_fallback_rate	Fallbacks fired / stale docs in corpus	= 1.0	HIGH
p95_hybrid_latency_ms	95th pct latency, HYBRID path	$\leq 800\text{ms}$	MEDIUM
p95_migration_latency_ms	95th pct latency, MIGRATION path	$\leq 2000\text{ms}$	MEDIUM
llm_cost_per_query_usd	Average LLM cost across all paths	$\leq \$0.003$	LOW

Goal Mode Agentic Eval Metrics

Metric	Definition	Gate	Priority
goal_parse_accuracy	Goals correctly parsed (all deps extracted, no invented versions) / total	≥ 0.95	CRITICAL
dependency_extraction_accuracy	Deps correctly extracted / total deps in manifest	≥ 0.98	CRITICAL
task_plan_completeness	Tasks created / deps requiring migration check	= 1.0	HIGH
task_order_correctness	Task plans with correct priority ordering / total plans	≥ 0.95	HIGH
recovery_policy_correctness	Recovery actions matching policy rules / total recovery events	= 1.0	CRITICAL
aggregate_verdict_correctness	Correct aggregate verdicts / total goal runs	= 1.0	CRITICAL
trace_completeness_rate	Task runs with all required fields / total runs	≥ 0.98	HIGH

Metric	Definition	Gate	Priority
unbounded_retry_rate	Tasks with retry_count > 2 / total tasks	= 0.0	CRITICAL
llm_planning_violation_rate	Tasks where LLM used for planning/routing/recovery / total	= 0.0	CRITICAL
wrong_version_answer_rate	Inherited from CAT-2; applies to all task-level retrieval	= 0.0	CRITICAL

Embedding Regression Gate

Every CI run re-runs the golden eval suite against the current index. If version accuracy drops below 1.0, or P@5 on CAT-6 drops by more than 5pp vs baseline, the CI gate fails and deployment is blocked. This ensures embedding model changes cannot silently degrade version correctness.

17. Demo Narrative

30-Second Pitch

DevPulse is an agentic developer change-intelligence system. You can query it about specific dependency migrations — it returns version-correct, conflict-checked, grounded answers. Or you can give it a migration goal and a dependency manifest — it decomposes the goal into an ordered task queue, runs each dependency through a version-aware retrieval and conflict-detection pipeline, applies deterministic recovery when evidence is missing or conflicts are found, and produces an auditable SAFE/RISKY/BLOCKED aggregate migration plan. Every step is logged. The agent does not reason freely — it orchestrates correctness-preserving tools across multiple tasks.

2-Minute Architecture Walkthrough

DevPulse has two entry modes sharing all infrastructure. In Query Mode, a single migration query flows through an 11-layer pipeline: parsing, version extraction, hybrid BM25+pgvector+RRF retrieval with hard version pre-filtering, deterministic conflict detection across 9 conflict types, SAFE/RISKY/BLOCKED migration verdict, conditional LLM synthesis with post-generation grounding validation, and programmatic citation assembly. The LLM is only invoked at Layer 8, only on HYBRID/MIGRATION paths, and only when there are no unresolved conflicts.

In Goal Mode, a dependency manifest and goal text enter a GoalParser that extracts structured dependency records without inventing version information — reading target versions from a controlled demo registry. A DependencyDeltaDetector classifies version deltas and breaking-change risk by semver rules. A TaskPlanner creates a priority-ordered queue — major version jumps first. A TaskExecutor loops over the queue, calling the full Query Mode pipeline for every task. When a task returns BLOCKED, a rule-based RecoveryDecider applies a bounded retry policy. A PlanSummaryReporter aggregates all outcomes into a goal-level SAFE/RISKY/BLOCKED verdict with a staged migration recommendation and a list of blockers.

5-Minute Query Mode Demo Walkthrough

- [1] Show EA-01 (ingest_summary.json): 3 sources, chunk count, metadata fill rate >= 95%.

- [2] Run simple lookup: 'what is the rate limit in v2?' — show BM25 path, no LLM called, citation with version_tag=v2 in < 100ms.
- [3] Run version comparison: 'what changed in authenticate() between v2 and v3?' — show hybrid retrieval, RRF scores in EA-09, conflict_alert for same_api_different_behavior, RISKY verdict.
- [4] Run adversarial trap: 'how does authenticate() work?' — no version hint — show conflict flag, BLOCKED verdict, evidence-only response.
- [5] Show EA-15 (golden_eval_results.json): version accuracy = 1.0, wrong-version answer rate = 0.0 (target acceptance gates).
- [6] Show EA-23 (cost_latency_report.json): SIMPLE P95 < 100ms, HYBRID P95 < 800ms, avg LLM cost < \$0.003.
- [7] Show EA-21 (langfuse_trace_export.json): LLM span with prompt, retrieved chunks, token count, latency.
- [8] Explain what you would add next: reranker, Slack integration, structured diff view — and why each is left for a future version.

5-Minute Goal Mode / Agentic Demo Walkthrough

- [1] Show EA-25 (agent_goal_parse_sample.json): 5-dep manifest, parsed goal, ambiguity flag on dep with missing target version.
- [2] Show EA-26 (dependency_delta_report.json): two major/HIGH, one minor/MEDIUM, one patch/LOW, one unknown. target_source = 'controlled_demo_registry' for resolved deps.
- [3] Show EA-27 (agent_task_plan.json): priority-1 tasks first, reason_for_priority populated for each.
- [4] Show EA-28 (agent_task_execution_trace.json): Task 1=RISKY; Task 2=BLOCKED (CRITICAL conflict, synthesis suppressed); Task 3+4=SAFE; Task 5=BLOCKED (missing evidence). query_id links each task to query_audit_log.
- [5] Show EA-29 (recovery_decision_log.json): Task 2 → skip_and_escalate (CRITICAL conflict, no retry). Task 5 → retry_with_related_version_evidence → second run finds related evidence → task improves from BLOCKED to RISKY with caveats. retry_count=1. No SAFE verdict issued — exact full coverage still incomplete. Proves RecoveryDecider is not only an escalation logger.
- [6] Show EA-30 (plan_summary_report.json): aggregate_verdict=BLOCKED because critical-priority dependency (Task 2) remains BLOCKED. Staged recommendation: Task 3+4 (SAFE) can proceed independently; Task 5 (RISKY) can proceed only with caveats and reviewer approval; Task 2 must be resolved before full migration.
- [7] Show EA-31 (agentic_eval_results.json): all metrics at target gates, unbounded_retry_rate=0, llm_planning_violation_rate=0.
- [8] Run show_agentic_demo.py live: full terminal output in under 3 minutes.
- [9] Explain v4 scope: repo inspection, code patches, test execution, PR generation — intentionally out of scope for v3.

Final Interview Positioning Summary

DevPulse v3.0 is not a generic agent demo. It is a production-simulated agentic developer change-intelligence system whose core differentiator is bounded multi-step goal execution while preserving version correctness, deterministic conflict detection, grounded synthesis, auditability, and evidence artifacts. The 37-day lifecycle, 32 artifacts, 19 failure scenarios, and agentic eval suite distinguish it from both a portfolio prototype and a free-form agent.

18. Must-Have / Should-Have / Optional / Never-Cut

Core System Must-Haves

MUST-HAVE — Query Mode Core
• Ingestion pipeline with structural chunking (preserves change entry boundaries) and metadata enrichment • Hybrid BM25 + pgvector + RRF retrieval with hard version pre-filter • Deterministic conflict detection — all 9 types • LLM synthesis conditional on conflict_flag and verdict (not BLOCKED) • Post-generation validation: ungrounded API name detection → fallback_event + synthesis suppressed • Programmatic citation assembly (not LLM-generated) • Migration decision report: SAFE / RISKY / BLOCKED with evidence requirements • Four Query Mode audit schemas: query_audit_log, retrieval_traces, conflict_alerts, fallback_events • Golden eval suite: 50+ queries, 10 categories, adversarial wrong-version traps • Langfuse traces + Sentry error monitoring • seed_devpulse.py generating Days 1–30 lifecycle • EA-01 to EA-24 (all 24 Query Mode evidence artifacts)

Agentic Must-Haves

MUST-HAVE — Goal Mode Agentic Layer

- GoalParser with deterministic manifest parsing and ambiguity flagging
- Manifest support: package.json, requirements.txt, pasted manifest text (must-have)
- dependency_target_registry.json — controlled demo registry for target version resolution
- DependencyDeltaDetector with semver-based delta and risk classification
- TaskPlanner / TaskQueue with deterministic priority ordering
- TaskExecutor loop over full Query Mode pipeline per task (no bypass of any Query Mode layer)
- RecoveryDecider with rule-based recovery policy and retry caps (max 2 per task)
- PlanSummaryReporter with aggregate verdict logic and staged migration recommendation
- Five Goal Mode agentic audit schemas
- EA-25 to EA-32 (all 8 Goal Mode evidence artifacts)
- F-11 to F-19 (all 9 Goal Mode failure/recovery scenarios including F-19 successful recovery)
- Agentic eval harness with 10 target acceptance gates
- seed_devpulse_v3.py generating Days 31–37 lifecycle
- show_agentic_demo.py producing EA-32

SHOULD-HAVE

- Reviewer resolution workflow for detected conflicts
- Source freshness tracking and stale-doc fallback (source_freshness schema)
- Embedding replacement without downtime (blue/green index swap)
- CI regression gate: version accuracy = 1.0 enforced on every commit
- Cost and latency dashboard
- Dependency criticality tagging (user-supplied critical runtime path flag)
- Goal-level cost and latency reporting (sum of per-task costs)
- pyproject.toml and pom.xml manifest support (optional in v3.0)

OPTIONAL

- Structured version-to-version diff view
- Upgrade impact summary: what breaks, what is additive
- Slack interface for migration queries or plan notifications
- Reranker (cross-encoder) for top-10 → top-5 re-ranking
- Additional ingestion sources: npm docs, GitHub releases
- Web-based plan summary report UI
- Repo inspection (scan actual repo files for dependency usage)
- Code patch proposal generation (v4 scope)
- Test execution integration (v4 scope)
- PR generation / GitHub / GitLab integration (v4 scope)

NEVER-CUT

- Version filtering correctness — wrong-version answer rate = 0.0 in all queries and all tasks
- LLM-last rule — LLM never plans, routes, recovers, classifies risk, or assembles citations
- Deterministic conflict detection — even a 3-type subset is non-negotiable
- Programmatic citations — citations built from chunk metadata, never generated by LLM
- Audit trail — at minimum query_audit_log + retrieval_traces
- Adversarial wrong-version traps in eval suite
- Deterministic recovery — RecoveryDecider is rule-based, never LLM-driven
- No unbounded loops — retry_count capped at 2 per task; unbounded_retry_rate = 0.0
- Staged migration recommendation — SAFE/RISKY/BLOCKED per-task action logic in PlanSummaryReporter
- Trace visibility — EA-28 and EA-32 must show the full agent loop
- Aggregate verdict correctness — F-16 must produce BLOCKED correctly

19. Final Deliverable Checklist

Ingestion & Indexing

- [] Ingest pipeline runs end-to-end on 3 sources
- [] BM25 + pgvector index built and verified
- [] Metadata fill rate $\geq 95\%$ (version_tag, change_type, has_deprecation) — EA-01
- [] EA-02: chunk_metadata_sample.json shows all fields populated

Retrieval Correctness

- [] Version pre-filter enforced: EA-08 shows 0 wrong-version chunks
- [] RRF fusion improves P@5 vs BM25-only by $\geq 10\%$ — EA-07
- [] EA-09: retrieval_traces_sample.json shows full trace for 10 queries

Conflict Detection

- [] All 9 conflict types implemented in ConflictDetector
- [] Conflict alerts created for all 9 injected conflict scenarios — EA-10
- [] BLOCKED verdict fires on CRITICAL severity conflicts
- [] EA-11: conflict_alerts_schema.sql matches implemented DDL

LLM Synthesis & Citation

- [] LLM called only on HYBRID/MIGRATION paths with conflict_flag=False or resolved
- [] Post-generation validation catches ungrounded API names
- [] fallback_event logged on ungrounded output; synthesis discarded — EA-18
- [] Citations built programmatically — EA-14
- [] EA-13: synthesis_grounding_report.json grounding rate ≥ 0.95

Migration Decision Report

- [] SAFE / RISKY / BLOCKED logic implemented and tested
- [] All 12 output fields populated for every migration report
- [] EA-12: migration_decision_samples.json shows 5 reports covering all verdict types

Evaluation

- [] Golden eval suite: 10 categories, 50+ queries
- [] Version accuracy = 1.0 in CI gate — EA-15
- [] Wrong-version answer rate = 0.0 — EA-16
- [] Conflict detection rate ≥ 0.90 — EA-15

Observability & Audit

- [] All 4 Query Mode audit schemas implemented and populated during lifecycle
- [] Langfuse traces captured for all LLM calls — EA-21
- [] Sentry error monitoring active — EA-22
- [] EA-17 through EA-24 all present and non-empty

Query Mode 30-Day Lifecycle & Demo

- [] seed_devpulse.py runs end-to-end
- [] show_demo_report.py produces terminal evidence summary
- [] All 24 Query Mode evidence artifacts present and non-empty
- [] EA-24: devpulse_demo_report.txt is coherent and complete

Goal Mode Agentic Layer

- [] dependency_target_registry.json authored with controlled demo versions
- [] GoalParser correctly parses well-formed manifest AND flags ambiguous inputs
- [] DependencyDeltaDetector classifies version deltas without LLM — EA-26
- [] TaskPlanner produces deterministic priority-ordered task queue — EA-27
- [] TaskExecutor calls full Query Mode pipeline per task; query_id populated — EA-28
- [] RecoveryDecider applies correct policy for all 4 BLOCKED conditions — EA-29
- [] F-19 successful recovery (BLOCKED → RISKY) demonstrated in EA-29
- [] PlanSummaryReporter produces aggregate verdict + staged recommendation — EA-30
- [] All 5 Goal Mode agentic schemas implemented and populated

Goal Mode Evaluation & Demo

- [] Agentic eval harness covers 10 metrics — all gates met — EA-31
- [] unbounded_retry_rate = 0 asserted in EA-31
- [] llm_planning_violation_rate = 0 asserted in EA-31
- [] show_agentic_demo.py runs end-to-end; EA-32 produced
- [] All 8 Goal Mode evidence artifacts (EA-25 to EA-32) present and non-empty

20. Resume and Interview Positioning

Resume Bullets (use 2 of 3)

1 Built DevPulse v3.0, a production-simulated agentic developer change-intelligence system combining version-aware BM25+pgvector+RRF retrieval, deterministic conflict detection across 9 conflict types, LLM-last grounded synthesis, SAFE/RISKY/BLOCKED migration decision reports, and bounded agentic orchestration over dependency migration goals — with goal parse accuracy, aggregate verdict correctness, and trace completeness captured in EA-31 after implementation.

2 Designed a 17-node system (11-layer Query Mode pipeline + 6-component Goal Mode agentic layer) with 12 audit schemas, 32 evidence artifacts, 19 failure/recovery scenarios, and a 37-day simulated operating lifecycle — preserving LLM-last orchestration, hard version pre-filtering, and deterministic conflict detection throughout both modes.

- 3** Extended a version-aware migration decision system with bounded agentic recovery policies (retry cap = 2 per task, 40% blocked = goal-level halt, deterministic RecoveryDecider); agentic eval harness asserts unbounded_retry_rate = 0 and llm_planning_violation_rate = 0 — with results captured in agentic_eval_results.json and agentic_demo_report.txt after implementation.

Interview Q&A;

Q: Why not just use a generic RAG chatbot over the docs?

A: A generic chatbot retrieves the most semantically similar chunk regardless of version. For migration decisions, returning the right answer for the wrong version is worse than no answer — it generates confident incorrect guidance. DevPulse treats version correctness as a hard constraint enforced at the retrieval layer, not a soft preference left to the LLM.

Q: How do you prevent wrong-version answers?

A: Three mechanisms in combination: (1) hard SQL pre-filter on version_tag before scoring, so wrong-version chunks never enter the ranking; (2) ConflictDetector checks retrieved chunks for version_deprecation_conflict and same_api_different_behavior — any CRITICAL conflict forces a BLOCKED verdict; (3) the adversarial eval suite (CAT-7, 10 trap queries) asserts wrong_version_answer_rate = 0.0 as a non-negotiable gate.

Q: Why LLM-last?

A: Because version correctness cannot be probabilistic. If the LLM plans retrieval, filters versions, detects conflicts, or assembles citations, it may hallucinate version numbers, invent API names, or prioritise the wrong version's documentation. LLM-last ensures all deterministic correctness work completes before the LLM is optionally invoked for compression and natural-language synthesis.

Q: How does conflict detection work without an LLM?

A: Deterministically, using chunk metadata. If two retrieved chunks for the same API name have different version_tag values and different change_type values, that fires a same_api_different_behavior conflict. All 9 conflict types are detected via metadata comparisons — not model reasoning. This means conflict detection is fast, inspectable, and cannot hallucinate a false positive.

Q: Why is Goal Mode agentic and not just batch processing?

A: Batch processing applies the same fixed operation to all inputs. Goal Mode makes decisions during execution that affect subsequent steps: RecoveryDecider changes the retrieval strategy based on what ConflictDetector found; TaskPlanner orders tasks based on risk signals from DependencyDeltaDetector; the aggregate verdict is computed dynamically from task outcomes. The loop responds to its own outputs — that is the definition of agentic behaviour.

Q: Why not let the LLM plan?

A: If the LLM plans the task sequence, it may create tasks for the wrong version, infer non-existent dependencies, or prioritise based on hallucinated risk signals. Every planning decision in Goal Mode is deterministic: manifest parsing extracts only what is in the file; version deltas are computed from semver comparison; task priority follows an explicit rule table. LLM planning would violate the core version-correctness guarantee.

Q: How does RecoveryDecider work?

A: It applies a rule table deterministically. The trigger condition determines the recovery action: missing docs → `retry_with_cross_source_expansion`; HIGH/CRITICAL conflict → `skip_and_escalate` immediately (no retry); low evidence → `retry_with_related_version_evidence`; vector store down → BM25-only fallback. No LLM involvement. The recovery may expand evidence scope but never lowers the version-correctness bar — exact requested-version coverage is still required for a SAFE verdict.

Q: How do you prevent infinite loops?

A: Explicitly, with policy caps. Each task has a maximum `retry_count` of 2. If still BLOCKED after two retries, `capped_by_policy = true` and the task is escalated. At the goal level, if $\geq 40\%$ of tasks are BLOCKED, the entire goal run is halted. The agentic eval harness asserts `unbounded_retry_rate = 0` — verifiable from EA-31.

Q: What happens when one dependency is BLOCKED?

A: It depends on the dependency's priority. If it is priority-1 (critical), the aggregate verdict cannot be SAFE regardless of all other task outcomes. `PlanSummaryReporter` adds it to `do_not_proceed_blockers`. The staged migration recommendation lists SAFE and RISKY dependencies as potentially proceeding independently, with explicit caveats for RISKY tasks and a full stop for BLOCKED critical-priority tasks.

Q: Why does this appeal to AI-forward CEOs and applied AI teams?

A: AI-forward leaders are not looking for people who can use AI tools. They want people who can build AI systems that reduce human effort at scale in a way that is inspectable and trustworthy. DevPulse v3 demonstrates that: a system that takes a messy input, structures it deterministically, executes a multi-step correctness-preserving plan, handles failures without human intervention where possible, escalates when it cannot — and produces an auditable output a team can act on. That is AI system ownership, not AI usage.

21. Red-Team Review

Risk: The retrieval is sophisticated-looking but not genuinely strong

Measure P@5 on real queries before touching synthesis. If BM25-only outperforms hybrid on exact API lookups, the vector index adds noise. Verify version filter correctness before claiming version accuracy targets.

Mitigation: EA-07 must show RRF improvement $\geq 10\%$ vs BM25-only. EA-08 must show 0 wrong-version chunks. Run both checks before any demo.

Risk: Conflict detection is cosmetic, not useful

All 9 types must fire on injected scenarios. If detection only fires on obvious same-version contradictions, it does not cover the hard case (changelog vs reference doc disagreement). At least 2 subtle conflict types must appear in the golden eval.

Mitigation: EA-10 must cover all 9 types. CAT-10 in the golden eval must include cross-source conflict queries.

Risk: Eval dataset is too easy

If every golden query has a single unambiguous answer, the eval does not test hard cases. Must include: multi-version queries, adversarial wrong-version traps, no-confident-answer queries, and cross-source conflict queries.

Mitigation: CAT-7 (10 adversarial traps) and CAT-9 (5 no-confident-answer queries) are non-negotiable. EA-16 must show 0 wrong-version answers.

Risk: Version correctness is talked about more than enforced

Run EA-08 check before every demo. If wrong-version answer rate > 0.0, fix the filter first. This is the highest-priority correctness gate in the system.

Mitigation: Do not demo without EA-08 showing 0 wrong-version chunks and EA-16 showing 0 wrong-version answers.

Risk: The agentic layer looks like a loop over v2 queries

Differentiation requires: (a) goal decomposition before retrieval, (b) deterministic task ordering based on risk signals, (c) rule-based recovery that changes retrieval strategy mid-run, (d) aggregate verdict logic across tasks, (e) visible execution trace.

Mitigation: EA-30 must show at least one recovery action that changed the retrieval strategy AND produced a different outcome than the first attempt (F-19 BLOCKED → RISKY recovery). Without this, the recovery layer cannot be distinguished from an escalation logger.

Risk: GoalParser is just JSON parsing

The differentiation is ambiguity detection: flagging missing target versions, partial manifests, unrecognised ecosystems, and unresolvable version strings — without inventing anything.

Mitigation: EA-25 must include at least one dep with missing target version flagged correctly AND one well-formed dep extracted correctly. The contrast is the interview signal.

Risk: Aggregate verdict logic is trivial

If aggregate verdict is just 'any BLOCKED = BLOCKED,' it does not demonstrate nuanced judgment. The staged migration recommendation and the F-16 mixed-verdict scenario are what elevate it.

Mitigation: EA-30 must show a mixed-verdict run (F-16) with a staged recommendation listing which deps can proceed, which need caveats, and which block full migration.

Risk: Rule-based recovery is not impressive

LLM-driven recovery is actually worse for version-sensitive systems — it may bypass version constraints.
Deterministic recovery is a feature: bounded, inspectable autonomy over free-form LLM judgment.

Mitigation: The interview Q&A; (Section 20) frames this argument. *recovery_policy_correctness = 1.0 in EA-31 proves the policy was applied correctly every time.*

What to Cut If Time Gets Tight

Cut in this order: (1) Slack/UI integrations, (2) reranker, (3) structured diff view, (4) additional ingestion sources, (5) user feedback capture, (6) pyproject.toml / pom.xml manifest support, (7) dependency criticality tagging, (8) goal-level cost/latency reporting, (9) agentic eval harness (manual verification instead), (10) show_agentic_demo.py (present EA-28 + EA-30 directly).

Do NOT cut: hybrid retrieval core · version filtering · conflict detection · grounded synthesis · observability · eval harness · RecoveryDecider with retry caps · staged migration recommendation.
The irreducible core: ingestion → metadata-aware hybrid retrieval → version conflict detection → grounded synthesis → citation assembly → audit schemas → evaluation harness → GoalParser → DependencyDeltaDetector → TaskPlanner → TaskExecutor → RecoveryDecider → PlanSummaryReporter → EA-28 + EA-30.

22. Recommendation Before Execution Planning

This section covers high-level build sequencing and rationale. A separate execution plan document will detail day-by-day tasks and milestones. Build in correctness-first order: each phase ends with a verifiable artifact proving that phase's functionality actually works.

Pre-condition for Goal Mode: DevPulse Query Mode must be fully implemented and all 24 Query Mode evidence artifacts (EA-01 to EA-24) must be present before beginning Goal Mode development.
seed_devpulse.py must run end-to-end before seed_devpulse_v3.py is built.

Week 1: Ingestion Correctness

Focus: Ingest 3 sources, structural chunking, metadata enrichment. BM25 + pgvector index built. Metadata fill rate >= 95% asserted. Version pre-filter implemented and tested. EA-01 to EA-05 produced.

Exit gate: *Version accuracy = 1.0 on 20 exact lookup queries (validated by version_filter_audit.json / EA-08)*

Week 2: Hybrid Retrieval + Conflict Detection

Focus: Hybrid BM25 + pgvector + RRF implemented. ComplexityRouter routes to SIMPLE/HYBRID/MIGRATION. ConflictDetector implements all 9 types. conflict_alerts schema and logging implemented. EA-06 to EA-11 produced.

Exit gate: *Conflict detection fires on all 9 injected scenarios (validated by conflict_detection_report.json / EA-10)*

Week 3: LLM Synthesis + Migration Reports

Focus: LLM synthesizer with post-generation grounding validation. Migration decision report (SAFE/RISKY/BLOCKED). Citation assembler (programmatic). Fallback events logging. EA-12 to EA-14 produced.

***Exit gate:** Synthesis grounding rate ≥ 0.95 on 15 migration queries (validated by `synthesis_grounding_report.json` / EA-13)*

Week 4: Eval + Observability + 30-Day Lifecycle

Focus: Golden eval suite: 50+ queries, 10 categories, adversarial traps. CI regression gate. Langfuse + Sentry + all 4 Query Mode audit schemas. `seed_devpulse.py` generates full Days 1–30 lifecycle. EA-15 to EA-24 produced.

***Exit gate:** `show_demo_report.py` runs end-to-end; all 24 artifacts present; all thresholds met (EA-24)*

Week 5 (Days 31–33): Goal Parsing + Task Planning

Focus: GoalParser, DependencyDeltaDetector, TaskPlanner. `dependency_target_registry.json` authored. Schemas: `agent_goals`, `agent_tasks`.

***Exit gate:** EA-27 exists; task priority ordering correct; no invented versions in EA-25*

Week 6 (Days 34–35): Task Execution + Recovery

Focus: TaskExecutor loop + RecoveryDecider. Schemas: `agent_task_runs`, `recovery_actions`. F-19 successful BLOCKED→RISKY recovery demonstrated.

***Exit gate:** EA-28 with `query_id` populated per task. EA-29 shows correct recovery actions. `unbounded_retry_rate` = 0.*

Week 7 (Days 36–37): Aggregation + Demo + Eval

Focus: PlanSummaryReporter, agentic eval harness, `show_agentic_demo.py`. Schema: `plan_summary_reports`. F-16 mixed-verdict scenario covered in EA-30.

***Exit gate:** EA-30 shows `aggregate_verdict` correctly applied over mixed-outcome goal run. EA-31 all agentic eval gates met. EA-32 runs end-to-end in under 3 minutes.*

23. Honest Boundary Statement

Does NOT Claim	Does Claim
Real enterprise deployment	Serious system design with justified tradeoffs
Real users or production traffic	Controlled simulation with realistic operating conditions

Does NOT Claim	Does Claim
Private production data	Reproducible artifacts from a fresh environment
Autonomous production decisions	Bounded agentic execution with deterministic recovery
LLM-driven planning or recovery	Deterministic goal decomposition and rule-based recovery policy
Unbounded autonomous loops	Retry-capped, policy-governed task execution (cap = 2)
Live npm / PyPI / Maven registry integration	Controlled demo registry (dependency_target_registry.json)
Real GitHub PR generation or test execution	Inspectable plan artifact with staged migration recommendation
Regulatory approval or security sign-off	Production-inspired operating discipline with full auditability
Implementation metrics (claimed before artifacts exist)	Target acceptance gates — become verifiable only after EA-15, EA-16, EA-31 exist

24. Revision History

Version	Date	Notes
v1.0	2026 Q1	Initial PRD: LLM-last architecture, hybrid retrieval, metadata enrichment, evaluation framework concept
v2.0	April 2026	Major upgrade: 30-day simulated lifecycle, 24 evidence artifacts, 10 failure scenarios, conflict_alerts schema, SAFE/RISKY/BLOCKED migration reports, adversarial eval, audit schemas
v2.1	April 2026	Cleanup: production-simulated terminology, artifact-safe resume bullets, target acceptance gate labeling, LLM-last rule made explicit, Section R renamed
v2.2	April 2026	Cover fix; real/simulated/controlled distinction clarified in Section C
v3.0 Upgrade PRD	April 2026	Agentic orchestration layer added: GoalParser, DependencyDeltaDetector, TaskPlanner, TaskExecutor, RecoveryDecider, PlanSummaryReporter. 5 new schemas. EA-25 to EA-32. F-11 to F-18. Days 31–37 lifecycle. Agentic eval harness.
v3.0 Clean	April 2026	Cleanup pass: fixed tuple artifact in preservation rules, renamed retry_with_relaxed_version_filter → retry_with_related_version_evidence, corrected manifest scope (must-have vs optional), added dependency_target_registry.json, added F-19 successful recovery, removed staged migration from cut list
v3.0 Consolidated	April 2026	Full consolidated master PRD: v2.2 core and v3.0 agentic layer merged into single unified document. Appendix-style language removed. 37-day lifecycle unified. EA-01 to EA-32 unified. F-01 to F-19 unified. Schemas, evaluation, demo, resume/interview, and red-team review all merged.

Source documents: DevPulse PRD v2.2 (April 2026) and DevPulse PRD v3.0 Agentic Orchestration Upgrade — Clean (April 2026). This consolidated master PRD supersedes both source documents as the single source of truth for DevPulse v3.0.